

ARL-Tangram：释放智能体强化学习中的资源效率

Bangjun Xiao^{†‡*} Yihao Zhao^{†‡*} Xiangwei Deng^{†‡} Shihua Yu[‡]
 Yuxing Xiang^{†‡} Huaqiu Liu[‡] Qiyang Wang[‡] Liang Zhao[‡]
 Hailin Zhang^{‡§} Xuanzhe Liu[†] Xin Jin^{†◊} Fuli Luo^{‡◊}

[†] 北京大学计算机学院

[‡] 小米 LLM-Core

Abstract

智能体强化学习 (RL) 已经成为云集群中的一种关键新型工作负载，它使大语言模型 (LLM) 能够通过真实世界交互来解决复杂问题。然而，与传统 RL 不同，智能体 RL 需要大量位于主训练集群之外的外部云资源，例如用于代码执行的 CPU、用于奖励模型的 GPU 等。现有智能体 RL 框架通常依赖静态过量预留，也就是把资源长期绑定在寿命较长的轨迹上，或者按任务隔离部署，从而带来严重的资源低效。

我们提出动作级编排思想，并将其实现为 ARL-Tangram，一个统一的资源管理系统，用于支持细粒度的外部资源共享与弹性调度。ARL-Tangram 采用统一的动作级建模和弹性调度算法，在满足异构资源约束的同时最小化动作完成时间 (ACT)。进一步地，我们还为不同类型、不同拓扑特征的资源设计了异构资源管理器，以高效支持动作级执行。真实世界智能体 RL 任务上的评测表明，ARL-Tangram 可将平均 ACT 最多降低 4.3 $\times$ ，将 RL 训练 step 时长最多加速 1.5 $\times$ ，并最多节省 71.2% 的外部资源。该系统已经部署并用于支持 MiMo 系列模型的训练。

1 引言

智能体强化学习 (RL) 已经在处理现实世界复杂问题方面展现出显著性能提升，例如 AI 编程 (Dou et al., 2024; Le et al., 2022)、深度搜索 (Qi et al., 2025; Zheng et al., 2025) 与具身智能 (Wang et al., 2023; Zitkovich et al., 2023)。与仅依赖内部知识的传统大语言模型 (LLM) 不同，智能体会通过 shell 命令、API 或设备操作等外部工具主动与真实世界交互 (Cao et al., 2025)。为了支撑这种能力，智能体 RL 已成为后训练流程中的关键组成部分，也是云集群中的重要工作负载 (Xiao et al., 2026)。

与传统 RL 不同，智能体 RL 对外部资源有大量需求，例如 AI 编程所需的 CPU、奖励模型所需的 GPU，以及网页浏览所需的 API 配额。这些位于主训练集群之外的外部资源引入了额外的管理需求。虽然这与云资源管理问题有相似之处，但智能体场景对外部资源的高效管理提出了更高要求，主要体现在两个方面。第一，外部调用延迟与 RL 效率紧密耦合。缺乏有效的外部资源管理时，调用可能变慢甚至阻塞，不仅会拖慢训练，还可能导致整个训练流程失败。第二，外部

* 共同贡献，部分工作完成于小米实习期间。

§ 项目负责人。

◊ 共同通讯作者。

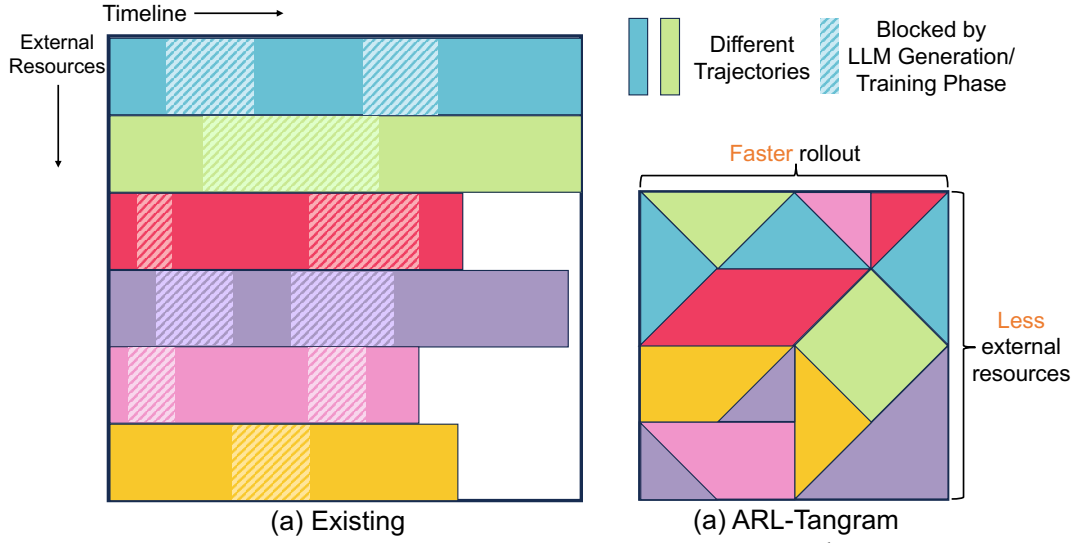


Figure 1 现有方法与 ARL-Tangram 的对比。¹

资源带来的额外成本不容忽视。智能体 RL 的调用模式通常具有突发性，因此为了维持较低的调用延迟，往往不得不预留大量资源。所以，提高外部资源利用率并降低对应成本非常重要。

然而，现有 RL 框架 (Fu et al., 2025; Sheng et al., 2025) 往往忽略了 RL 训练与外部资源使用之间的耦合关系。这些框架默认采用静态资源预留，导致外部资源出现严重浪费与低效。过量预留主要发生在两个层面（图 1）。其一，在轨迹层面，系统通常会在一条轨迹的整个生命周期中长期保留专用资源，即便这些资源只在少数时刻被调用。例如在 AI 编程中，环境平均仅有 47% 的生命周期真正被访问，剩余时间所占用的 CPU 都被浪费了。其二，在 RL 任务层面，不同任务通常需要特定的外部服务，这些服务会部署在相互隔离的资源上 (Jiang et al., 2025; Microsoft, 2025)。但由于外部调用模式波动很大，这些资源长期处于低利用率状态。总之，这种静态管理限制了系统并发度，增加了调用排队延迟，并拖慢了 RL 训练。

为了解决这些问题，我们提出 **动作级**调度，把外部资源管理粒度从原先的轨迹级或任务级下沉到更细的动作级，即原子调用的粒度。我们把寿命较长的环境或服务占用拆解为动作级占用，并把相同资源类型对应的资源统一池化给动作使用。除此之外，这种细粒度管理还允许系统进行弹性资源分配，从而降低动作执行延迟。正如图 1 所示，当有两个 RL 任务、四条轨迹访问同类外部资源时，动作级调度相比现有方法可以通过缓解过量预留来减少所需外部资源，并通过弹性分配来加速 rollout。

不过，实现动作级调度并不简单，主要有三个原因。第一，面向多种外部资源对动作进行统一编排本身就很复杂。单个动作可能同时依赖多种资源类型，而不同动作的弹性与执行模式又各不相同，因此需要一个通用抽象模型。第二，调度器必须面对强延迟敏感工作负载。动作持续时间可能只有微秒级，这意味着调度决策窗口极短，需要足够轻量的算法来处理高并发和突发流量。第三，如何统一且高效地管理具有不同特征与拓扑结构的异构外部资源同样具有挑战。

因此，我们设计了 ARL-Tangram (**A**gentic **R**einforcement Learning **T**angram)，一个动作级资

¹该图仅用于解释 ARL-Tangram 的核心直觉，实际模式并不严格是三角形。

源管理系统，用来统一编排这些外部资源调用。首先，它采用统一的动作建模来描述具有异构资源需求和成本的动作。系统把每个动作表示为向量化的资源成本，其中包含 CPU、GPU、内存以及 API 配额等多种约束。更关键的是，这一建模还纳入了弹性刻画，使系统能够识别弹性动作，并计算在分配更多资源时动作时延如何下降。借助这一建模，ARL-Tangram 能把不同类型的动作归一化成统一格式供调度器处理。

ARL-Tangram 的核心是一个面向最小化动作完成时间 (ACT) 的弹性资源调度算法。考虑到更短的动作执行时间会直接提升端到端智能体 RL 训练效率，我们提出了一个带贪心驱逐机制的启发式调度算法，它根据统一建模与实时系统状态来编排动作。这个贪心驱逐策略会动态确定资源分配方式，从而避免过于激进或过于保守的分配导致次优 ACT 和较差的 RL 效率。

在调度器之外，ARL-Tangram 还为异构资源设计了专门的资源管理器。每类管理器都提供相应机制，用于高效地实现 **Breakdown**，也就是在每个动作执行后释放资源，同时保留环境或服务状态并在再次调用时快速恢复；以及 **Pool**，也就是在缓解碎片化并提升并行效率的原则下进行资源分配。尽管资源在特征和拓扑上高度异构，这些管理器仍通过统一接口向调度器暴露能力，从而让调度算法对底层异构性保持透明。

我们将 ARL-Tangram 实现为一个独立于 RL 框架、外部调用类型和外部资源类型的系统。这一设计带来了更强的通用性与更低的接入复杂度。我们在真实世界智能体 RL 训练任务上评估 ARL-Tangram，结果表明它可将平均 ACT 最多降低 4.3 $\times$ ，把 RL 训练 step 时长最多加速 1.5 $\times$ ，并最多节省 71.2% 的外部资源。该系统已被用于支持 MiMo 系列模型的 RL 训练。

总的来说，本文贡献如下：

- 我们分析了智能体 RL 训练中外部资源过量预留这一关键问题，并把它归纳为轨迹内过量预留和 RL 任务内过量预留两类。
- 我们提出动作级调度，并将其实现为统一资源管理系统 ARL-Tangram，把管理粒度从轨迹级下沉到动作级，从而实现细粒度共享与弹性调度。
- 我们设计了统一的动作建模以及面向最小化 ACT 的弹性调度算法，并结合异构资源专用管理器进一步提升外部资源利用率与训练效率。
- 我们在真实 RL 训练任务上评估 ARL-Tangram，证明它能够显著提升智能体 RL 的端到端效率，并降低外部资源成本。

2 背景

本节首先在 §2.1 中介绍智能体 RL 的训练流程；随后在 §2.2 中讨论外部资源管理问题；再在 §2.3 中分析过量预留现象；最后在 §2.4 中总结我们看到的机会与挑战。

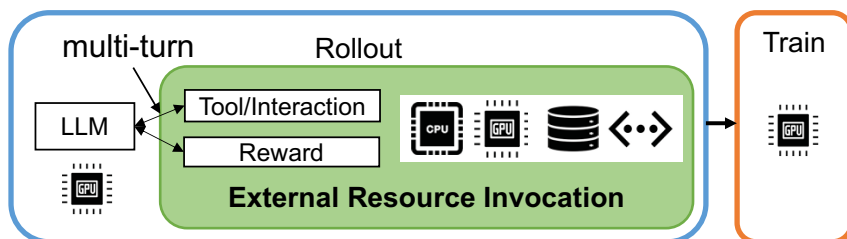


Figure 2 智能体 RL 的一次训练 step。

2.1 智能体 RL 训练

智能体在 AI 编程、深度搜索和具身智能等现实任务中表现出明显的性能优势 (Dou et al., 2024; Qi et al., 2025; Wang et al., 2023)。与传统 LLM 不同，智能体 LLM 会通过 shell 命令、API 或设备操作等外部工具与真实世界交互。这种工具使用方式使得智能体 LLM 的 RL 流水线与传统 LLM 显著不同。

如图 2 所示，给定一个 batch 的样本后，RL 框架首先使用智能体 LLM 处理这些样本并收集执行轨迹，这一阶段在 RL 中通常称为 *rollout*。在 rollout 期间，框架通常遵循 ReAct 模式 (Yao et al., 2022)：LLM 根据输入提示决定如何使用工具，随后 RL 框架执行相应工具调用并接收环境观测。轨迹结束后，系统会基于偏好信号（如 RLHF (Ouyang et al., 2022) 和 LLM-as-a-judge (Bai et al., 2022)）或基于真值信号（如 RLVR (Guo et al., 2025)）计算奖励，并据此训练 LLM。

在一条轨迹内部，智能体 RL 往往会多轮重复 ReAct 模式，以提升模型的工具使用能力。虽然 LLM 的生成与训练主要消耗 GPU，但工具调用会依赖 CPU、GPU、存储和网络等异构资源。从 RL 框架视角看，这些位于训练系统之外的资源统称为 **外部资源**。频繁使用外部资源正是智能体 RL 的典型特征。例如 AI 编程会反复通过 CPU 执行 shell 命令或编辑文件，而 DeepSearch 会频繁访问网站并消耗 API 配额。除此之外，轨迹结束时的奖励计算往往也依赖外部资源，例如 LLM-as-a-judge 常被部署为独立模型服务，而 AI 编程任务的奖励评估需要运行大量测试用例。凡是发生在 RL 框架之外、需要调用额外资源的操作，都可以统一称为 **外部资源调用**。

2.2 外部资源管理

智能体 RL 训练依赖外部资源来承载环境、工具或奖励服务，因此对这些外部资源的管理会直接影响 RL 效率与成本。

第一，RL 任务的效率高度依赖外部资源使用是否合理。如果外部资源不足，或者资源分配不合适，外部调用的执行时长就会增加。根据 §2.1 中的常见工作流，这些外部调用都位于 rollout 的关键路径上，因此更长的调用意味着更慢的 rollout，也意味着更慢的 RL 训练。图 3(a) 展示了同一 RL 任务在不同外部资源规模下的对比：当资源不足（仅 0.5×）时，平均 ACT 和 step 时长都明显高于资源充足（1×）的情况。更严重的是，若外部调用失败过多，整个 RL 训练任务也可能失败 (Jiang et al., 2025)。

第二，外部调用会带来额外资源成本，因此降低这部分成本同样很关键。实际中，智能体 RL 往往需要部署 CPU 集群来构建环境，或部署专用 GPU 运行奖励服务，这些都会产生额外费用。例

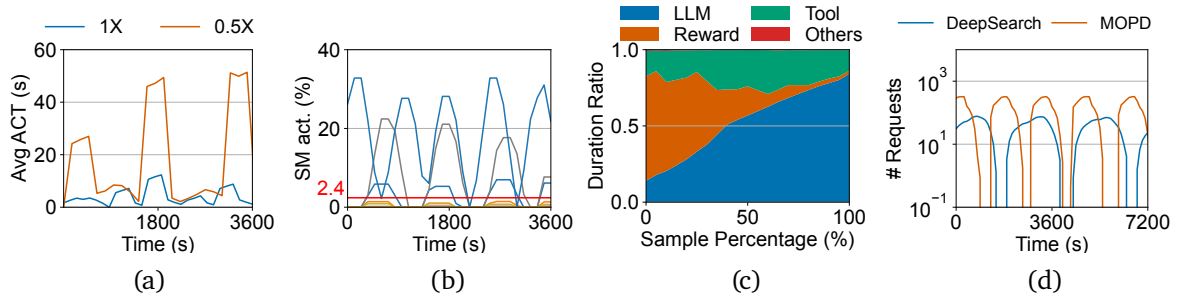


Figure 3 (a) 在 1× 与 0.5× 外部资源规模下的平均 ACT；(b) MOPD 中 12 个奖励服务的 SM 活跃度；(c) 代码智能体 rollout 时长占比；(d) 两个智能体 RL 任务中的外部调用次数。

如 MOPD (Xiao et al., 2026) 这样的蒸馏流程，为了给单个 RL 任务打分，可能需要数十个教师模型和数百张 GPU。然而这些外部 GPU 并没有被充分利用。图 3(b) 给出了 12 个教师模型对应 GPU 的 SM 活跃度，可以看到它在时间维度与模型维度上都大幅波动，平均值甚至不到 3%，说明外部资源浪费极为严重。

2.3 外部资源的过量预留

轨迹内部的过量预留。在智能体 RL 中，LLM 生成与外部调用会在轨迹内部交替进行，但现有系统通常会在 rollout 开始时就创建环境并长期保留相应外部资源，直到整条轨迹结束才释放。结果是，这些被预留的资源在大部分时间里都处于空闲状态，本可以被其他轨迹复用。以 AI 编程为例，图 3(c) 表明被浪费的外部资源可占到 53%。在有限外部资源条件下，这种轨迹内部过量预留会显著限制系统并发、增加工具调用排队延迟，并阻碍系统为计算密集型操作动态追加资源，从而拉长 rollout 时间、降低 RL 框架整体效率，也降低训练集群中 GPU 的利用率。

RL 任务内部的过量预留。在任务层面，不同 RL 任务往往需要不同的外部服务来完成交互或奖励计算，因为服务类型和配置都可能不同 (Jin et al., 2025a; Li et al., 2025; Xiao et al., 2026)。例如，不同任务使用的奖励模型可能架构或参数不同，因此通常会被部署在彼此隔离的资源上 (Jiang et al., 2025; Microsoft, 2025)。但这些服务并不是一直都处于高负载状态。对于单个 RL 任务来说，在训练阶段甚至根本不会发生外部调用，因此预留出来的外部服务会长时间闲置。图 3(d) 展示了 DeepSearch 与 MOPD 两个真实智能体 RL 任务中的外部调用次数，其数量变化可以达到三个数量级，因此会造成严重的外部资源浪费。

2.4 机会与挑战

动作级调度带来的机会。为了应对上述过量预留导致的低效，我们提出在 **动作级**管理外部资源。这里的 *action* 指一次原子化的外部资源调用，在其执行期间，轨迹中不会穿插 LLM 生成或其他动作。不同于让每条轨迹各自占有资源并独立完成调用，动作级方案把所有动作提交到统一系统中进行集中调度。系统会把寿命较长的环境或服务占用拆解成动作级占用，也就是 **Breakdown**；同时，它会对同类资源进行池化，并在动作之间弹性分配，也就是 **Pool**。通过 **Breakdown** 与 **Pool**，系统能够在空闲时释放或缩减资源，从而消除过去由于轨迹内 LLM 生成阶段或任务内训练阶段造成的过量预留。除此之外，动作级调度还带来了更灵活、更细粒度的资源分配能力。当

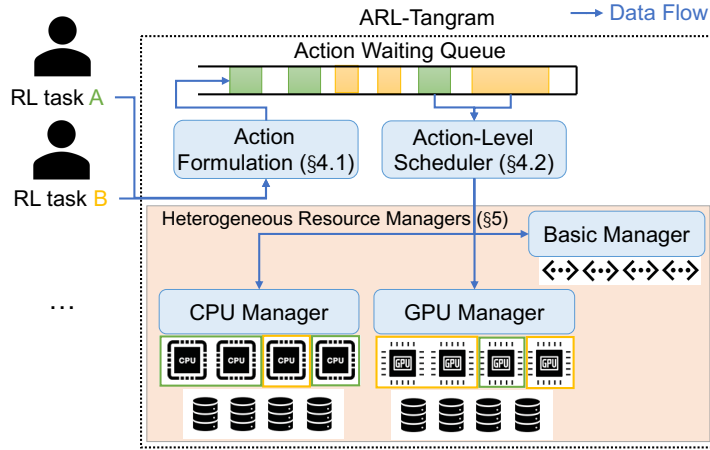


Figure 4 ARL-Tangram 的系统总览。

资源充足时，它可以为弹性动作动态分配更多资源单元，提高并行度（DoP），从而缩短执行时长，进一步提升整体 RL 训练效率。

概括来看，把资源管理粒度从轨迹或 RL 任务下沉到单个动作，可以带来两点好处：(i) 通过跨轨迹、跨任务复用，减少长空闲期造成的资源闲置；(ii) 通过动态扩缩容，降低与外部环境交互的延迟。

挑战。 尽管动作级调度带来了明确机会，但它也面临多项挑战。首先，不同动作可能同时依赖多种资源，并且不同资源在容量、约束和可扩展性上都不同，因此需要统一的动作建模。其次，调度算法需要在极短时间窗口内完成决策，否则调度本身的开销就会抵消收益。最后，不同资源具有截然不同的拓扑和执行特性，例如 CPU 集群、GPU 集群和外部 API 配额都不能用同一种方式管理，因此还需要与统一调度器协同工作的异构资源管理器。

3 系统架构

为了应对当前智能体 RL 训练中多层次的过量预留问题，无论是轨迹内部还是 RL 任务之间，我们提出了统一的动作级平台 ARL-Tangram。该架构将外部资源管理从主 RL 训练集群中解耦出来，把控制粒度从寿命较长的轨迹下沉到单个原子交互，也就是动作。

如图 4 所示，系统位于多种 RL 任务与异构外部资源之间，充当统一中介。其工作流程遵循标准化执行周期：(1) 动作提交：在 rollout 阶段，当智能体 LLM 调用工具或触发依赖外部资源的奖励计算时，它会向 ARL-Tangram 提交请求。(2) 统一建模与排队：ARL-Tangram 把不同的外部资源调用翻译成统一动作，并提交到统一动作队列。(3) 弹性调度：调度器在外部资源约束下调度等待队列中的动作，并与专门的资源管理器（如 CPU、GPU、API）协同，根据实时系统状态和动作建模结果进行弹性资源分配。(4) 动作执行：确定资源单位后，动作会在异构资源管理器分配到的资源上独立执行。(5) 结果回传与观测：动作执行完成后，结果会被传回 RL 框架，使其继续进行下一步 LLM 生成。

ARL-Tangram 主要包含三大组件，即动作建模模块 (§4.1)、动作级调度器 (§4.2) 和异构资源

管理器 (§5)。这套架构的基础思想是把每一次原子调用都视作一个动作。因此，建模模块以统一方式刻画各种动作的成本和性能弹性；调度器则在资源约束下判断动作能否被调度，并以最小化动作完成时间 (ACT) 为目标为其弹性分配资源。这样，ARL-Tangram 能减少 LLM 等待外部资源返回的时间，进一步加速 RL 训练。获得调度决策后，异构资源管理器负责高效地分配与共享资源。借助统一资源接口，包括 Basic Manager、CPU Manager 和 GPU Manager 在内的管理器使用各自领域特定的技术来提升系统鲁棒性、资源利用率和动作执行速度。

4 统一的动作级建模与调度

本节先在 §4.1 中给出动作级统一建模，覆盖异构外部资源与多样化工具类型；然后在 §4.2 中介绍我们的弹性资源调度算法。

4.1 动作建模

向量化资源成本建模。为了实现统一的动作级调度，首先需要对资源成本进行清晰而统一的表示。给定一个动作 a_i ，它可能同时消耗多种资源类型。例如，DeepSearch 任务中的一个动作可能会并行或串行地访问多个网站，而每个网站都有独立的 API 配额，因此可以被视作不同的资源类型；AI 编程任务中的一个动作则既依赖 CPU 也依赖内存。因此，我们将动作成本表示为多维向量 $C_i = (c_{i,0}, c_{i,1}, \dots, c_{i,k-1})$ ，其中 k 表示由 ARL-Tangram 管理的资源类型数量。该向量使调度器可以联合考虑多种资源约束。向量的每个维度分别对应一种资源消耗，包括 CPU、GPU、内存、显存、网络带宽以及外部 API 配额等。

更重要的是，单个资源维度上的成本并不是静态值。对于对资源数量敏感的弹性动作，其成本通常表示为从最小可行值到最大可行值的范围。更复杂的情况是，一些多 GPU 动作只允许离散资源单位，例如 1、2、4、8。因此， C_i 中的每个 $c_{i,j}$ 都附带自己的可行资源集合。这种建模方式使调度器能够在不破坏可行性的前提下灵活调整资源分配，并在调度时利用先验缩小搜索空间。

弹性建模。对弹性动作，我们额外引入参数来描述其弹性行为。对于某一特定资源，其弹性被建模为：当分配 m 个资源单位时，动作可获得一个介于 0 和 1 之间的弹性比例。文中采用如下形式：

$$a.getDur(m) = \frac{T_{ori}}{E(m) * m} \quad (0 < E(m) \leq 1), \quad (1)$$

其中 T_{ori} 表示动作在单个资源单位下的执行时长。为了简化建模与优化，我们假设一个动作的所有资源中，只有一种资源类型主导其弹性。例如，对 GPU 动作而言，显存是执行必要条件，但真正决定时延的通常是并行度。因此，动作的弹性可以视作关于关键资源类型使用量的映射。

执行时长建模。为了服务于后续的弹性调度算法，动作建模中除了弹性以外，还需要显式纳入执行时长。因此，我们为可剖析的动作记录其原始执行时长，并按单个资源单位归一化。配合弹性定义，系统便能够估计不同资源分配下的时间代价，这对优化调度决策、在性能与资源效率之间做平衡是必要的。

Algorithm 1 弹性资源调度算法。

- 1: 候选动作先按照 FCFS 从等待队列中选出
 - 2: 每个动作先分配其最小可行资源
 - 3: 按关键弹性资源将动作分组
 - 4: 对每组动作分别执行弹性分配
 - 5: 通过近似目标函数比较不同驱逐与分配方案
 - 6: 输出本轮真正进入执行的动作及其资源分配
-

4.2 弹性资源调度

在动作级资源分配与调度的设定下，我们需要一个面向有限资源条件的弹性调度算法来保证效率。我们把动作完成时间（ACT）作为优化目标。类似于作业完成时间（JCT），ACT 可分解为两部分：(1) 动作排队等待被调度的时间；(2) 动作实际执行时间。因此，调度目标可以写成最小化所有动作 ACT 总和：

$$\text{Objective: minimize ACTs} = \sum_{0 \leq i < N} (T_i^q + T_i), \quad (2)$$

其中 N 表示调度时等待队列中的动作数， T_i 为第 i 个动作的实际执行时长， T_i^q 为其排队时间。

具体到调度问题，它包含两个相互耦合的子问题：(1) 确定等待队列中动作的执行顺序；(2) 确定每个动作应分配多少资源单位。这两个问题都不容易。为了缩小搜索空间，并围绕本文关注的核心现象“资源过量预留”展开，我们主要优化第二个问题，同时对第一个问题采用简单但稳定的 First-Come First-Served (FCFS) 策略。由于所有动作都位于各自轨迹的关键路径上，智能体 RL 对 starvation 十分敏感，因此 FCFS 是一个稳妥选择。当然，我们的调度框架也可以与其他排序策略结合。

第二个问题本质上是一个经典难题：在有限资源下为多个弹性作业分配资源，并最小化总 JCT，该问题已知是 NP-hard (Bruno et al., 1974)。又由于调度窗口极短，同时资源类型和作业模式都高度异构，因此我们采用启发式方法。总体上，我们会先根据资源最小需求选出一组候选动作，然后按其关键弹性资源分组，再分别执行弹性分配。对于可缩放动作，我们使用带目标近似的贪心驱逐算法：先用最小资源计算初始目标值，然后逐步移除候选集合尾部动作，并把其资源回收给前面的动作；若目标值下降，则继续驱逐，否则停止。为估计目标，我们把候选集合的 ACT 精确计算，而把等待队列剩余动作的 ACT 通过近似方式估计，从而控制调度开销。

对于既可伸缩又已知弹性与时长信息的动作，我们提出了基于目标近似的贪心算法。它先假设所有候选动作均采用最小资源，再不断尝试驱逐队尾动作并把资源重新分给保留下来的动作；一旦继续驱逐不再降低目标值，算法便停止。候选集合内部的资源分配通过一个拓扑无关的动态规划（DP）算法求解，而等待队列剩余动作则通过顺序插入完成堆的方式进行近似。为了提高近似精度，系统还引入了参数 $depth$ ，允许最靠前的剩余动作探索更多资源分配选项。实践中， $depth = 2$ 或 3 已经足够。

对不可伸缩资源或弹性未知的动作，算法则直接选择这些候选动作，并仅为其分配最小所需资源，同时保证所有向量化资源约束满足。

5 异构资源管理器

虽然我们给出了统一抽象和调度算法，但要真正实现 **Breakdown** 与 **Pool** (§2.4)，仍然需要资源特定的管理策略。具体而言，**Breakdown** 关注如何在释放资源的同时保留执行状态或支持快速恢复，也就是高效上下文切换；同时还要支持每个动作的弹性并行度。**Pool** 则关注如何设计资源分配方式以缓解碎片化并提高并行效率，从而为统一的弹性调度框架提供支撑。正如 §2.4 所述，不同资源在特征和拓扑结构上差异很大，因此我们为不同资源类型设计了不同管理器。

5.1 Basic Resource Manager

Basic Manager 面向那些不需要特殊拓扑建模、也不需要复杂上下文切换机制的资源。它实现统一调度器所需的基础能力，例如资源申请、释放、容量可见性与排队协调，是整个系统的最小实现单元。

5.2 基于 AOE 的 CPU Manager

CPU 资源管理器主要面向 AI 编程等使用环境执行、命令运行和测试计算的场景。

CPU Manager 中的 Breakdown。对 CPU 场景而言，Breakdown 的关键是：在动作结束后尽快释放 CPU 资源，同时保留环境状态，以便下一个动作到来时能够快速恢复，而不必重新构建整个执行环境。

CPU Manager 中的 Pool。在 Pool 方面，CPU Manager 通过感知节点内资源与任务需求，把相同类型的 CPU 资源池化给不同动作共享使用，并减少资源碎片，提升并发执行效率。对可扩展动作而言，它还能提高并行度以缩短执行时间。

5.3 基于 EOE 的 GPU Manager

GPU 资源管理器主要面向奖励模型、多模型服务和教师模型等依赖 GPU 的外部服务。

GPU Manager 中的 Breakdown。在 GPU 场景中，Breakdown 更加复杂，因为动作往往依赖模型恢复、缓存状态与张量并行配置。GPU Manager 的目标是在动作结束后回收 GPU 资源，同时尽量缩短后续恢复成本。

GPU Manager 中的 Pool。在 Pool 方面，GPU Manager 需要同时考虑模型并行度、显存、GPU 数量以及拓扑结构。它通过把资源池化给多个模型或服务共享，减少传统静态部署下的过量预留，并在高并发时依据弹性调度器决策动态调整资源占用。

6 评测

我们首先用多种智能体 RL 任务评估 ARL-Tangram 的 ACT 效率 (§6.2)，随后考察其随 RL batch size 与外部资源规模变化时的可扩展性 (§6.3)，最后分析 ARL-Tangram 的设计效果与系统开销 (§6.4)。

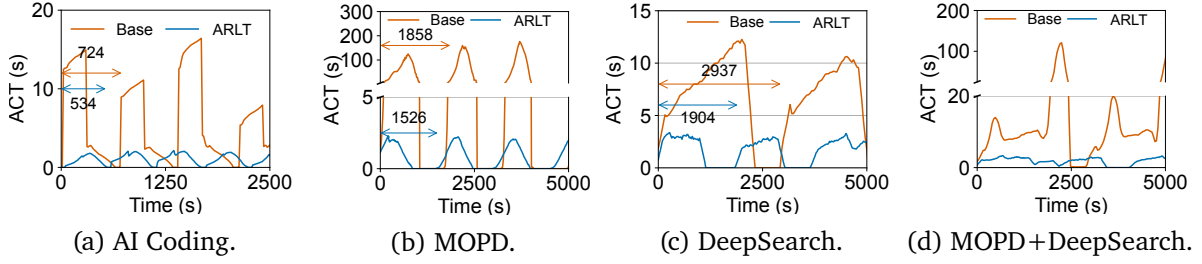


Figure 5 不同 RL 任务下的 ACT。图中数字和箭头表示 RL 任务的 step 时长。

6.1 实验设置

测试平台。测试平台由两部分组成：一部分是运行智能体 RL 框架的 GPU 集群，另一部分是由 ARL-Tangram 管理的一组异构外部资源。RL 框架部署在生产环境 GPU 集群上，最多包含 48 个 GPU 节点，每个节点配备 8 张基于 NVIDIA Hopper 架构的 GPU，并通过高带宽 NVLink 与 RDMA 网络互联。我们使用 VeRL (Sheng et al., 2025) 作为智能体 RL 框架，并为其扩展了外部资源调用、序列级和异步 *rollout* 能力。ARL-Tangram 同时部署在 CPU 与 GPU 集群上：CPU 集群包含 15 个节点，每个节点拥有 256 个 AMD CPU 核和 2.4 TB 内存；GPU 集群包含 5 个节点，每个节点拥有 8 张高端 NVIDIA GPU 和 3 TB CPU 内存。此外，ARL-Tangram 还管理若干带配额限制的外部 API 服务，例如 Google Search 和 PDF 解析服务。

工作负载。我们在 §6.2 中选取三类智能体 RL 任务作为工作负载。**AI Coding** 使用 CPU 构建隔离执行环境，环境会被间歇性调用，并用于计算奖励分数。我们采用一个面向生产环境、基于 SWEbench (Jimenez et al., 2024) 脚手架构造的内部数据集。**DeepSearch** 持续访问外部网站来辅助 LLM 问答，其数据集基于 BrowseComp (Wei et al., 2025)，奖励则由基于 LLM 的判别器在生成轨迹上进行计算，具体使用微调后的 GPT-OSS (Agarwal et al., 2025)。上述两个任务都采用 GRPO (Shao et al., 2024)。**MOPD** (Xiao et al., 2026) 在 rollout 阶段同时整合多个 RL 任务（包括智能体任务），并相对于对应教师模型计算轨迹对数概率以完成对齐。为了展示 ARL-Tangram 在缓解“RL 任务内部过量预留”方面的收益，我们还让 DeepSearch 与 MOPD 并发运行，因为二者都需要 GPU 来部署外部服务。

基线。由于没有现有系统能统一支持这三类工作负载，我们为 §6.2 中的实验分别设计了任务特定基线。对于 **AI Coding**，我们使用 Kubernetes 管理 CPU 集群。每条轨迹在执行开始时申请创建一个 pod，每个 pod 分配 0.5 个 CPU，以允许有限复用，同时设定最多 4 个 CPU 的上限以避免拥塞。对于 **MOPD**，我们使用 SGLang (Zheng et al., 2024) 在 GPU 集群上部署 9 个教师模型，每个模型分配 4 张 GPU 并使用 tensor parallelism (TP)。对于 **DeepSearch**，基线允许每条轨迹独立执行 API 调用，并在错误时最多重试 3 次，超时阈值为 600 秒。DeepSearch 的奖励服务也部署在 GPU 集群上，共 5 个副本，TP 度数为 8。对于“MOPD+Search”，共部署 10 个奖励服务，每个模型分配 4 张 GPU 且采用 TP。

指标。为了比较 ARL-Tangram 与基线的效率，我们主要测量平均 ACT。同时，我们还报告多个阶段的细粒度时长，包括 LLM 生成、工具调用和奖励计算。所有阶段时长都用 ARL-Tangram 的总时长归一化，因此 ARL-Tangram 的归一化时长总和为 1。

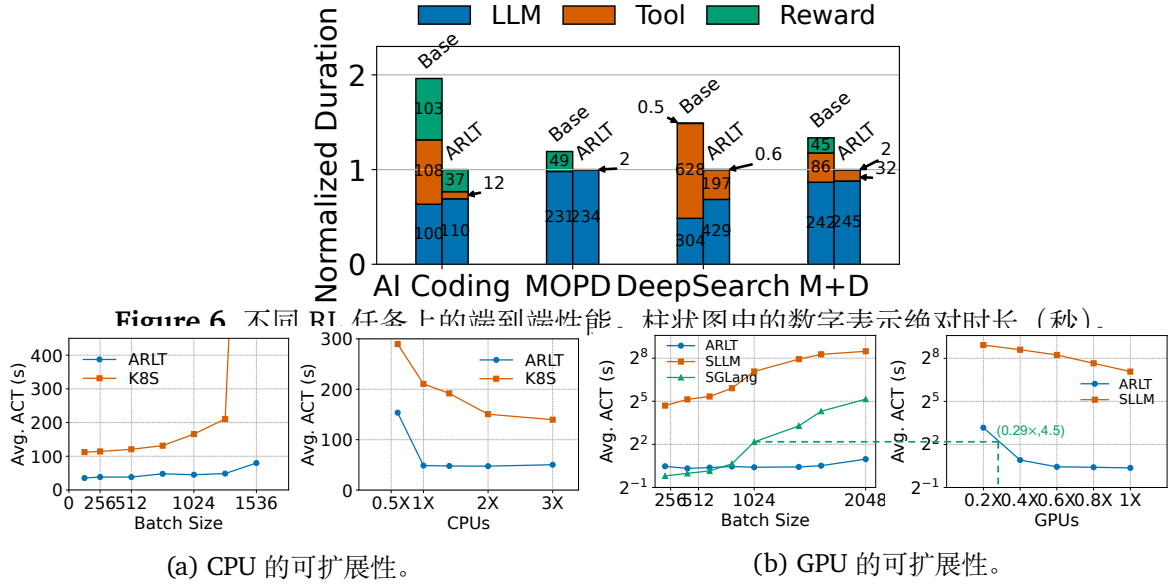


Figure 7 不同外部资源在 RL batch size 和资源容量变化下的可扩展性。

其他设置。本节中，不同工作负载的 RL 训练 batch size 与外部资源容量固定为：AI Coding 为 1280，MOPD 为 2048，DeepSearch 为 2048。RL 训练模型使用 Qwen3-32B (Yang et al., 2025) 与 MiMo-V2-Flash (Xiao et al., 2026)。此外，ARL-Tangram 的具体配置包括：调度算法中的 *depth* 设为 2；只对 CPU 上的奖励计算和 GPU 上的奖励模型推理预先做可扩展性与执行时长 profiling。

6.2 端到端性能

我们在多种工作负载上评估端到端性能，以展示 ARL-Tangram 的通用性与效率。图 5 给出了 RL 训练推进过程中、连续小时时间窗口内的平均 ACT。可以看到，在所有工作负载下，ARL-Tangram 的 ACT 都持续低于基线。这说明在给定相同外部资源条件下，ARL-Tangram 能更有效地处理突发工作负载，并通过缓解过量预留、提高外部资源利用率来降低 ACT。即便 DeepSearch 中的 API 调用本身不可扩展，对配额和并发的合理管理也能减少由于限流或超时引起的错误和重试，从而降低 ACT。

我们进一步报告了 10 个 RL 训练 step 的平均时长，即 **step duration**，以体现 ARL-Tangram 对端到端训练效率的提升。AI Coding 和 DeepSearch 的 step 时长分别获得了 1.4× 和 1.5× 的显著优化。对 AI Coding 而言，收益主要来自动作级调度，它提高了外部 CPU 的利用率；对 DeepSearch 而言，由于 API 调用本身不可扩展，step 时长的下降主要来自流量控制与更稳定的调用行为。总体上，ARL-Tangram 通过缩短外部调用等待时间和提升资源复用效率，提高了训练系统的端到端效率。

6.3 系统可扩展性

我们进一步通过同时放大并发度和资源容量来评估 ARL-Tangram 的有效性。重点考察那些动作本身可扩展的资源，也就是 CPU 和 GPU。我们测量每条轨迹的总 ACT，并在不同 **batch size** 和

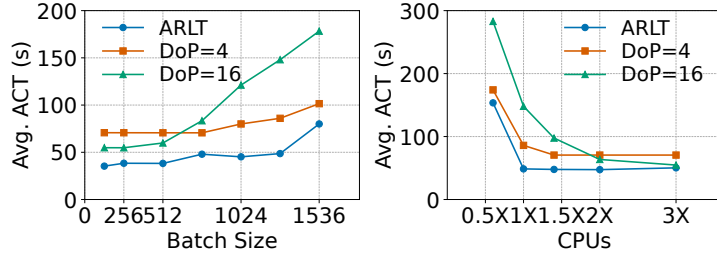


Figure 8 ARL-Tangram 调度策略的影响。

资源容量下取轨迹平均。由于采用 GRPO，因此每个 batch 中的轨迹数可以视作 RL batch size。

CPU 的可扩展性。在图 7(a) 中，所有实验都使用 5 个节点共 1280 个 CPU 核作为外部资源。随着 RL batch size 增大，ARL-Tangram 相比基线可将平均 ACT 降低 3.1× 到 27.7×。尤其在 batch size 为 1536 时，Kubernetes 基线由于资源不足，其控制平面开始频繁出现排队超时；而 ARL-Tangram 在该规模下仍能保持可接受的 ACT 增长，相比 batch size 为 128 时的 ACT 增幅不足 2×。右图固定 batch size 为 1280，在这一配置下基线仍比 ARL-Tangram 慢 1.89× 到 4.33×。当总 CPU 核数降为 768 时，收益有所减弱，因为资源更少会减少动作级调度跨轨迹复用的空间。

GPU 的可扩展性。除 SGLang 基线外，我们还引入了 ServerlessLLM (Fu et al., 2024) 作为另一个基线，它通过 Model-as-a-Service (MaaS) 抽象用更少 GPU 服务多个模型。与 CPU 场景类似，图 7(b) 左侧的实验都在 §6.1 所述的 5 节点 GPU 集群上完成。在高 batch size 下，ARL-Tangram 显著优于两个基线：当 batch size 为 2048 时，ARL-Tangram 相比 SGLang 可将平均 ACT 降低 18.1×，而 ServerlessLLM 在这种并发度下已无法提供可接受服务。作为参考，在 batch size 为 1024 时，ARL-Tangram 相比 SGLang 和 ServerlessLLM 分别快 3.4× 与 101.8×。需要指出的是，在低并发下，由于 ARL-Tangram 存在恢复开销，SGLang 的时延会略优一些。

图 7(b) 右侧则进一步说明，在固定 batch size 为 1024 的情况下，ARL-Tangram 在降低外部资源成本方面同样具有优势。ARL-Tangram 仅使用过量预留基线所需 GPU 数量的 29%，就能为 10 个奖励服务提供支持并实现相同的 ACT。虽然 ARL-Tangram 和 ServerlessLLM 都能用更少 GPU 部署多个模型，但由于 ServerlessLLM 缺少针对奖励服务的弹性重分配机制，且系统开销更高，因此其时延明显更差。

6.4 ARL-Tangram 设计分析

ARL-Tangram 调度策略的影响。我们在 AI Coding 工作负载上对弹性调度算法做了消融分析。由于 CPU 计算负载高度依赖 LLM 输出，随机性较强，我们基于一条真实 AI Coding RL 轨迹构建了一个 benchmark，并据此比较不同调度基线。在该任务中，只有奖励计算类动作具备 CPU 级可扩展性，因为它们执行时间长尾明显，也适合并行化。我们将动态 DoP 在 1 到 32 范围内变化的弹性调度算法，与两个固定 DoP 基线 (DoP=4 与 DoP=16) 进行比较。图 8 表明，弹性调度在所有设置下都优于固定 DoP 基线。其原因在于，它可以根据资源竞争情况动态调整资源分配策略。

工作负载 (bsz)	Coding (1280)	Coding (1536)	MOPD (2048)	MOPD (3072)
执行时长	0.975	1.206	0.621	0.705
排队时长	0.015	0.428	0.081	15.05
系统开销	0.024	0.036	0.201	0.240

Table 1 ACT 分解。

系统开销分析。我们进一步分解 AI Coding (CPU 密集) 和 MOPD (GPU 密集) 的 ACT, 以分析系统开销, 结果见表 1。可以看到, 即便在 AI Coding 的高拥塞场景下, 系统开销相对于执行时长仍然只占很小部分; 而对 GPU 场景中的 MOPD 而言, 恢复带来的额外开销更明显, 但这种开销不会随着更高并发或更严重排队而爆炸式增长, 因此系统在高并发智能体 RL 训练下仍然能保持稳定与高效。

建模精度。除了端到端收益, 我们还验证了 ARL-Tangram 建模的准确性。总体上, 执行时长与弹性建模为调度器提供了足够准确的相对比较依据, 使其能够在真实系统状态下做出有效的资源分配决策。

7 相关工作

RL 框架优化。LLM 强化学习的价值推动了大量 RL 框架优化研究 (Fu et al., 2025; Han et al., 2025; He et al., 2025; Hu et al., 2024; Lei et al., 2024; Shen et al., 2024; Sheng et al., 2025; Wang et al., 2025b; Yao et al., 2023; Zhong et al., 2025a,b)。OpenRLHF (Hu et al., 2024) 和 verl (Sheng et al., 2025) 分别探索了解耦与共置架构。后续一些框架又提出融合 (Zhong et al., 2025b)、流水化 (Zhong et al., 2025b)、流式执行 (Zhong et al., 2025a) 以及异步执行 (Fu et al., 2025) 等机制来提高系统效率。此外, 还有一些优化工作提出了请求级调度、负载均衡和 speculative decoding (Gao et al., 2025; Jin et al., 2025b), 以进一步加速 rollout 阶段。尽管这些工作很有价值, 但现有框架通常仍把外部调用视为次要问题, 要么忽略其资源管理, 要么默认采用静态过量预留。因此, 在轨迹更长、环境交互更复杂的智能体 RL 中, 会造成明显资源浪费。

智能体资源管理。随着智能体 RL 复杂度不断提高, 对工具与奖励所依赖外部资源的管理也变得更为重要。Mars (Zhu et al., 2026) 等近期工作已经开始关注奖励计算效率, 并把奖励计算部署为独立服务。Mars 利用“请求进入、批量输出”的特征, 把请求级延迟要求放宽为批级约束。尽管这对奖励计算有效, 但 ARL-Tangram 提供了更通用的方案: 它为所有类型的工具使用与奖励服务建立了统一的动作级平台, 覆盖 API 调用、代码执行与模拟器等多种场景。MegaFlow (Zhang et al., 2026)、SkyRL-Agent (Cao et al., 2025) 和 ROCK (Wang et al., 2025a) 则对智能体服务进行了抽象, 并借鉴云计算方法来管理工具。通过把工具的可扩展性与共享模式纳入建模, ARL-Tangram 能对工具调用和奖励服务做更有针对性的优化。

资源自动伸缩。自动伸缩是云计算的核心能力之一, 通常采用响应式、预测式或混合式方法 (Lorido-Botran et al., 2014; Wang et al., 2024; Wu et al., 2023, 2024; Zhang et al., 2024) 来满足服务等级目标 (SLO)。在 LLM 场景中, 相关系统 (Xiang et al., 2025; Yu et al., 2025; Zhang et al., 2025;

Zhao et al., 2025) 已经面向大规模推理工作负载优化了自动伸缩。针对 RL 场景, Mars (Zhu et al., 2026) 又引入了基于历史的伸缩与超时感知机制, 用于在 batch 边界上预测并调整资源需求。ARL-Tangram 在此基础上进一步提出了动作级弹性调度器, 以应对智能体 RL 动作的突发性和异构性。通过对动作可扩展性的显式建模, ARL-Tangram 可以最小化总 ACT, 从而加速 RL 训练过程。

8 结论

本文提出了 ARL-Tangram, 一个面向智能体 RL 的统一资源管理系统, 能够对异构外部资源进行细粒度共享与弹性编排。通过把外部交互建模为动作级对象, 并对其进行弹性调度, 同时为不同资源设计专门的资源管理器, ARL-Tangram 在提升总体训练效率的同时, 也改善了外部资源利用率。基于真实世界智能体 RL 工作负载的大规模评测表明, ARL-Tangram 能显著缩短训练 step 时长、降低平均 ACT, 并大幅减少外部资源成本, 使云环境中的智能体 RL 训练更加高效。

References

- S. Agarwal, L. Ahmad, J. Ai, S. Altman, A. Applebaum, E. Arbus, R. K. Arora, Y. Bai, B. Baker, H. Bao, et al. gpt-oss-120b & gpt-oss-20b model card. [arXiv preprint arXiv:2508.10925](#), 2025.
- Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, et al. Constitutional ai: Harmlessness from ai feedback. [arXiv preprint arXiv:2212.08073](#), 2022.
- J. Bruno, E. G. Coffman Jr, and R. Sethi. Scheduling independent tasks to reduce mean finishing time. [Communications of the ACM](#), 1974.
- S. Cao, D. Li, F. Zhao, S. Yuan, S. R. Hegde, C. Chen, C. Ruan, T. Griggs, S. Liu, E. Tang, et al. Skyr-rl-agent: Efficient rl training for multi-turn llm agent. [arXiv preprint arXiv:2511.16108](#), 2025.
- S. Dou, Y. Liu, H. Jia, L. Xiong, E. Zhou, W. Shen, J. Shan, C. Huang, X. Wang, X. Fan, et al. Step-coder: Improve code generation with reinforcement learning from compiler feedback. [arXiv preprint arXiv:2402.01391](#), 2024.
- W. Fu, J. Gao, X. Shen, C. Zhu, Z. Mei, C. He, S. Xu, G. Wei, J. Mei, J. Wang, et al. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning. [arXiv preprint arXiv:2505.24298](#), 2025.
- Y. Fu, L. Xue, Y. Huang, A.-O. Brabete, D. Ustiugov, Y. Patel, and L. Mai. ServerlessLLM: Low-Latency serverless inference for large language models. In [USENIX OSDI](#), 2024.
- W. Gao, Y. Zhao, D. An, T. Wu, L. Cao, S. Xiong, J. Huang, W. Wang, S. Yang, W. Su, et al. Rollpacker: Mitigating long-tail rollouts for fast, synchronous rl post-training. [arXiv preprint arXiv:2509.21009](#), 2025.
- D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, et al. Deepseek-rl: Incentivizing reasoning capability in LLMs via reinforcement learning. [arXiv preprint arXiv:2501.12948](#), 2025.
- Z. Han, A. You, H. Wang, K. Luo, G. Yang, W. Shi, M. Chen, S. Zhang, Z. Lan, C. Deng, et al. AsyncFlow: An asynchronous streaming RL framework for efficient LLM post-training. [arXiv preprint arXiv:2507.01663](#), 2025.
- J. He, T. Li, E. Feng, D. Du, Q. Liu, T. Liu, Y. Xia, and H. Chen. History Rhymes: Accelerating LLM reinforcement learning with RhymeRL. [arXiv preprint arXiv:2508.18588](#), 2025.
- J. Hu, X. Wu, W. Shen, J. K. Liu, Z. Zhu, W. Wang, S. Jiang, H. Wang, H. Chen, B. Chen, et al. OpenRLHF: An easy-to-use, scalable and high-performance RLHF framework. [arXiv preprint arXiv:2405.11143](#), 2024.

- D. Jiang, Y. Lu, Z. Li, Z. Lyu, P. Nie, H. Wang, A. Su, H. Chen, K. Zou, C. Du, et al. Verltool: Towards holistic agentic reinforcement learning with tool use. arXiv preprint arXiv:2509.01055, 2025.
- C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan. SWE-bench: Can language models resolve real-world github issues? In The International Conference on Learning Representations (ICLR), 2024.
- B. Jin, H. Zeng, Z. Yue, J. Yoon, S. Arik, D. Wang, H. Zamani, and J. Han. Search-rl: Training LLMs to reason and leverage search engines with reinforcement learning. arXiv preprint arXiv:2503.09516, 2025a.
- C. Jin, Y. Zhong, Z. Zhang, Y. Jiang, and Y. Zhu. Efficient rl for llms with dynamic and online speculative decoding. In Frontier AI Systems Workshop (FAISys), 2025b.
- H. Le, Y. Wang, A. D. Gotmare, S. Savarese, and S. C. H. Hoi. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. In Advances in Neural Information Processing Systems, 2022.
- K. Lei, Y. Jin, M. Zhai, K. Huang, H. Ye, and J. Zhai. PUZZLE: Efficiently aligning large language models through Light-Weight context switch. In USENIX ATC, 2024.
- X. Li, X. Sun, A. Wang, J. Li, and C. Shum. Cuda-ll: Improving cuda optimization via contrastive reinforcement learning. arXiv preprint arXiv:2507.14111, 2025.
- T. Llorido-Botran, J. Miguel-Alonso, and J. A. Lozano. A review of auto-scaling techniques for elastic applications in cloud environments. Journal of grid computing, 2014.
- Microsoft. Multi-agent reference architecture. <https://github.com/microsoft/multi-agent-reference-architecture>, 2025.
- L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. Advances in Neural Information Processing Systems, 2022.
- Z. Qi, X. Liu, I. L. Iong, H. Lai, X. Sun, J. Sun, X. Yang, Y. Yang, S. Yao, W. Xu, et al. WebRL: Training llm web agents via self-evolving online curriculum reinforcement learning. In The International Conference on Learning Representations (ICLR), 2025.
- Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, et al. Deepseek-math: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024.
- G. Shen, Z. Wang, O. Delalleau, J. Zeng, Y. Dong, D. Egert, S. Sun, J. Zhang, S. Jain, A. Taghibakhshi, et al. Nemo-aligner: Scalable toolkit for efficient model alignment. arXiv preprint arXiv:2405.01481, 2024.

- G. Sheng, C. Zhang, Z. Ye, X. Wu, W. Zhang, R. Zhang, Y. Peng, H. Lin, and C. Wu. HybridFlow: A flexible and efficient RLHF framework. In EuroSys, 2025.
- G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An open-ended embodied agent with large language models. arXiv preprint arXiv:2305.16291, 2023.
- W. Wang, X. Xu, W. An, F. Dai, W. Gao, Y. He, J. Huang, Q. Ji, H. Jin, X. Li, et al. Let it flow: Agentic crafting on rock and roll, building the rome model within an open agentic learning ecosystem. arXiv preprint arXiv:2512.24873, 2025a.
- Z. Wang, P. Li, C.-J. M. Liang, F. Wu, and F. Y. Yan. Autothrottle: A practical bi-level approach to resource management for SLO-targeted microservices. In USENIX NSDI, 2024.
- Z. Wang, T. Zhou, L. Liu, A. Li, J. Hu, D. Yang, J. Hou, S. Feng, Y. Cheng, and Y. Qi. DistFlow: A fully distributed RL framework for scalable and efficient LLM post-training. arXiv preprint arXiv:2507.13833, 2025b.
- J. Wei, Z. Sun, S. Papay, S. McKinney, J. Han, I. Fulford, H. W. Chung, A. T. Passos, W. Fedus, and A. Glaese. Browsecomp: A simple yet challenging benchmark for browsing agents. arXiv preprint arXiv:2504.12516, 2025.
- B. Wu, K. Qian, B. Li, Y. Ma, Q. Zhang, Z. Jiang, J. Zhao, D. Cai, E. Zhai, X. Liu, et al. Xron: A hybrid elastic cloud overlay network for video conferencing at planetary scale. In ACM SIGCOMM, 2023.
- Z. Wu, W.-L. Chiang, Z. Mao, Z. Yang, E. Friedman, S. Shenker, and I. Stoica. Can’t be late: optimizing spot instance savings under deadlines. In USENIX NSDI, 2024.
- Y. Xiang, X. Li, K. Qian, Y. Yang, D. Zhu, W. Yu, E. Zhai, X. Liu, X. Jin, and J. Zhou. Aegaeon: Effective gpu pooling for concurrent llm serving on the market. In ACM SOSP, 2025.
- B. Xiao, B. Xia, B. Yang, B. Gao, B. Shen, C. Zhang, C. He, C. Lou, F. Luo, G. Wang, et al. MiMo-V2-Flash technical report. arXiv preprint arXiv:2601.02780, 2026.
- A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, et al. Qwen3 technical report. arXiv preprint arXiv:2505.09388, 2025.
- S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models. In International conference on learning representations (ICLR), 2022.
- Z. Yao, R. Y. Aminabadi, O. Ruwase, S. Rajbhandari, X. Wu, A. A. Awan, J. Rasley, M. Zhang, C. Li, C. Holmes, et al. DeepSpeed-Chat: Easy, fast and affordable RLHF training of chatgpt-like models at all scales. arXiv preprint arXiv:2308.01320, 2023.

- M. Yu, R. Yang, C. Jia, Z. Su, S. Yao, T. Lan, Y. Yang, Y. Cheng, W. Wang, A. Wang, et al. λ Scale: Enabling fast scaling for serverless large language model inference. [arXiv preprint arXiv:2502.09922](#), 2025.
- D. Zhang, H. Wang, Y. Liu, X. Wei, Y. Shan, R. Chen, and H. Chen. BlitzScale: Fast and live large model autoscaling with O (1) host caching. In [USENIX OSDI](#), 2025.
- L. Zhang, M. Chen, R. Cao, J. Chen, F. Zhou, Y. Xu, J. Yang, L. Chen, C. Luo, K. Zhang, et al. Megaflow: Large-scale distributed orchestration system for the agentic era. [arXiv preprint arXiv:2601.07526](#), 2026.
- Z. Zhang, C. Jin, and X. Jin. Jolteon: Unleashing the promise of serverless for serverless workflows. In [USENIX NSDI](#), 2024.
- Y. Zhao, J. Chen, P. Sun, L. Li, X. Liu, and X. Jin. SeaLLM: Service-aware and latency-optimized resource sharing for large language model inference. [arXiv preprint arXiv:2504.15720](#), 2025.
- L. Zheng, L. Yin, Z. Xie, C. L. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, et al. Sglang: Efficient execution of structured language model programs. In [Advances in neural information processing systems](#), 2024.
- Y. Zheng, D. Fu, X. Hu, X. Cai, L. Ye, P. Lu, and P. Liu. Deepresearcher: Scaling deep research via reinforcement learning in real-world environments. [arXiv preprint arXiv:2504.03160](#), 2025.
- Y. Zhong, Z. Zhang, X. Song, H. Hu, C. Jin, B. Wu, N. Chen, Y. Chen, Y. Zhou, C. Wan, et al. StreamRL: Scalable, heterogeneous, and elastic RL for LLMs with disaggregated stream generation. [arXiv preprint arXiv:2504.15930](#), 2025a.
- Y. Zhong, Z. Zhang, B. Wu, S. Liu, Y. Chen, C. Wan, H. Hu, L. Xia, R. Ming, Y. Zhu, et al. Optimizing RLHF training for large language models with stage fusion. In [USENIX NSDI](#), 2025b.
- R. Zhu, M. Han, Y. Zhong, W. Xiao, X. Liu, and X. Jin. Towards efficient reward service for rlvr with request-level flexibility and batch-level constraint. In [USENIX NSDI](#), 2026.
- B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In [Conference on Robot Learning](#), 2023.

Algorithm 2 ACT 近似。

- 1: 保留原文伪代码结构，用于说明近似目标函数的计算流程
 - 2: 其核心思想是：精确计算本轮候选动作的 ACT，并对后续动作的 ACT 进行近似估计
-

Algorithm 3 与拓扑无关的 DPArrange

- 1: 保留原文算法主体所表达的动态规划思想
 - 2: 目标是在候选动作之间寻找满足离散资源约束的最优资源划分
-

A ACT 近似算法伪代码

B 统一 DPArrange 算法

本节给出 DPArrange 的附录说明。该算法负责在给定资源约束下，为候选动作求解离散资源分配，并可适配不同拓扑结构。

Algorithm 4 GPU 拓扑感知 DP 算子

- 1: 该算子在统一 DPArrange 之上，进一步处理 GPU 拓扑相关约束
 - 2: 包括并行度、设备组选择与资源布局等因素
-